



**MINISTRY OF EDUCATION, CULTURE AND RESEARCH  
OF THE REPUBLIC OF MOLDOVA**

**Technical University of Moldova**  
Faculty of Computers, Informatics and Microelectronics  
Department of Software Engineering and Automation

**Report**  
**Practical Work No. 1**

**Topic: Analysis of Sorting Algorithms Under Multiple Input Condition**

Made by:  
Lilia Martîn, FAF-253  
Dolinskaia Afina, FAF-253  
Bahnă Sebastian, FAF-253  
Fahed Maxim, FAF-253  
Laurențiu Palii, FAF-253  
Guțu Bogdan, FAF-253  
Gheorghilaș Daniel, FAF-253

Checked by:  
Department of Software and Automation Engineering,  
FCIM Faculty, UTM

## Abstract

Sorting algorithms are fundamental to computer science, forming the basis for optimizing search, indexing, and data organization in different applications. This research examines the performance and efficiency of five well-known algorithms: Tim Sort, Radix Sort, Quick Sort, Selection Sort, and Merge Sort, for different input sizes (from 100 to 1,000,000 elements) and distributions, including random, ascending, and descending order. The experimental methodology included implementing these algorithms in the C programming language and measuring execution time with nanosecond precision, as well as monitoring maximum memory usage using operating system calls in a standardized hardware environment. Quantitative results for the test with 1,000,000 random elements show that Quick Sort achieved the highest efficiency with a sorting time of 0.188 seconds, while Selection Sort proved impractical for large-scale data due to its quadratic complexity. Memory analysis showed that while Quick Sort and Selection Sort required only approximately 4.5–5.0 MB of peak memory, algorithms such as Merge Sort and Tim Sort required more than 13 MB due to the need for additional memory. The main conclusion of the research is that no single algorithm is universally optimal; performance depends largely on the characteristics of the input data, such as data order and memory constraints, with Tim Sort outperforming Quick Sort on partially sorted data and Quick Sort outperforming Tim Sort on random sets.

## 1 Introduction

Sorting in programming is the process of arranging the elements of a list or array in a specific order, such as ascending or descending, by number or lexicographically [1]. It is a fundamental concept in computer science, important for optimizing data search, combining data, and obtaining results that are understandable to humans. The need for sorting occurs in various contexts, from organizing library catalogs to optimizing search system results. Efficient sorting is crucial for indexing, searching, and organizing large data sets. At the same time, different algorithms are optimized for specific data types, computing environments, and application requirements. This diversity allows us to select the most effective algorithm depending on the specific context. Hardware capabilities such as computing power, memory capacity, and the availability of multiple processors can affect the performance of sorting algorithms. Algorithms that use parallel processing can benefit significantly from multi-core or distributed computing environments. In other words, the performance of an algorithm can increase significantly if its calculations are performed in parallel on multiple computing resources rather than sequentially on a single one. So, in our report, we are going to show and compare the benefits and efficiency of 5 of most common and often-used sorting algorithms, according to the above-mentioned aspects and features. Thus, we have chosen Radix sort, Merge sort, Quick sort, Tim sort and Selection sort.

## 2 Experimental Design

### 2.1 Objectives

Implement five sorting algorithms (TimSort, Radix, QuickSort, Selection, Merge) in C and measure their execution time and peak memory for input sizes 100, 1k, 10k, 100k, 1M on three distributions (random, ascending, descending). Ensure reproducibility (seeded inputs, high resolution timing, repeated trials) produce labeled plots (time and memory vs. size) compare results with theoretical complexity, and report recommendations and the two competition metrics for the 1,000,000 element case (sorting time; sorting time plus file output).

### 2.2 Evaluation Criteria

This section delineates the methodology employed to evaluate and compare the performance of the implemented sorting algorithms.

**Execution Time Measurement.** The principal performance metric in this study is execution time, measured as a function of the input size  $n$ . Execution time is indicative of the real-world efficiency of an algorithm and

facilitates empirical comparison under different input conditions. The experiments were implemented in the C programming language to ensure minimal runtime overhead and high measurement precision. Execution time was measured using:

```
timespec_get(&start, TIME_UTC);
timespec_get(&end, TIME_UTC);
```

from the `<time.h>` library. This function fills a `struct timespec`, which stores time in:

- seconds (`tv_sec`)
- nanoseconds (`tv_nsec`)

The total execution time was computed in nanoseconds using:

$$T(n) = (end.tv\_sec - start.tv\_sec) \times 10^9 + (end.tv\_nsec - start.tv\_nsec)$$

Results are reported primarily in nanoseconds and additionally converted to milliseconds (ms) for readability. Measuring time in nanoseconds increases precision, especially for small input sizes.

**Memory Consumption Evaluation.** Memory usage was evaluated using `getrusage(RUSAGE_SELF, &usage)`; from the `<sys/resource.h>` library. The metric used is `ru_maxrss` (maximum resident set size). This value denotes the maximum memory consumption of the process during execution and is reported in kilobytes (KB). This approach facilitates comparison between:

- in-place algorithms (e.g., Quick Sort and Selection Sort), and
- algorithms that require additional memory (e.g., Merge Sort, Radix Sort, and Tim Sort).

Although this metric reflects process-level peak memory rather than per-function allocation, it provides a consistent basis for comparative analysis.

**Repetitions and Result Aggregation.** In the current implementation, each configuration (algorithm + input size + input order) is executed once per program run. Therefore:

- no repeated trials are performed automatically,
- the reported value corresponds to a single measurement.

This approach simplifies experimentation but introduces minor variability due to operating system scheduling, background processes, and CPU cache state. For improved statistical reliability, future extensions could include multiple repetitions per configuration and computation of the average execution time:

$$\bar{T} = \frac{1}{k} \sum_{i=1}^k T_i$$

where  $k$  is the number of runs and  $T_i$  is the execution time of run  $i$ .

The algorithms are compared based on:

- execution time (nanoseconds and milliseconds),
- peak memory usage (KB),
- performance across three input conditions (ascending, descending, random),
- five input sizes: 100, 1,000, 10,000, 100,000, and 1,000,000 elements.

This methodology ensures a structured and reproducible empirical comparison of sorting algorithm performance under multiple input conditions.

## 2.3 Testing Environment

All sorting experiments were conducted on a **MacBook Pro 2020** using the following hardware and software configuration:

- **Processor (CPU):** Intel Core i5 (10th generation), base frequency 2.0 GHz, 4 physical cores and 8 logical threads
- **Memory (RAM):** 16 GB LPDDR4X, 3777 MHz
- **Graphics:** Intel Iris Plus Graphics

- **Operating System:** macOS Sequoia 15.7.4
- **Compiler:** Apple clang version 14.0.0
- **Programming Language:** C (ISO C11 standard)
- **Compilation Flags:** `-O2 -Wall` (code optimization enabled and all important compiler warnings activated)

**Input Data Generation.** The experimental evaluation was performed using pre-generated input files containing integer arrays of sizes 100, 1,000, 10,000, 100,000, and 1,000,000 elements. For each size, three different input distributions were considered:

- ascending order (best or near-best case for some algorithms),
- descending order (worst-case behavior for certain algorithms),
- random order.

Random input files were generated using a fixed seed to ensure reproducibility across multiple executions.

**Execution Time Measurement.** Sorting time was measured using the `timespec` structure and the `timespec_get` function with `TIME_UTC`, providing nanosecond-level resolution. The measured interval includes only the execution time of the sorting algorithm itself and excludes file input and output operations.

**Memory Usage Measurement.** Peak memory consumption was estimated using the `getrusage` system call with the `RUSAGE_SELF` flag. The reported value corresponds to the maximum resident set size (`ru_maxrss`) observed during program execution and is presented in kilobytes.

All experiments were executed under minimal background system load in order to reduce external interference and improve measurement consistency.

Second experiments (The Fastest Sorting Algorithm) were conducted on a **ASUS VIVOBOK** with the following hardware and software configuration:

- **Processor (CPU):** AMD Ryzen 5 (mobile series), base frequency  $\sim 2.1\text{--}2.3$  GHz, 6 physical cores and 12 logical threads
- **Memory (RAM):** 8 GB DDR4
- **Graphics:** Integrated AMD Radeon Graphics with 2 GB shared VRAM
- **Operating System:** Windows 11 (64-bit)
- **Compiler:** GCC MinGW version 14.0.2
- **Programming Language:** C (ISO C11 standard)
- **Compilation Flags:** `-O2 -Wall` (code optimization enabled and all important compiler warnings activated)

### 3 Sorting Algorithms

#### Algorithm 1: TimSort

Main idea:

Combine insertion sort and merge sort.  
First sort small blocks (runs) using insertion sort.  
Then repeatedly merge these runs until the entire array is sorted.  
Designed to take advantage of already partially sorted data.

Pseudocode:

```
1  FUNCTION insertionSort(arr, left, right)
2      FOR i FROM left+1 TO right
3          temp = arr[i]
4          j = i - 1
5          WHILE j >= left AND arr[j] > temp
6              arr[j+1] = arr[j]
7              j = j - 1
8          arr[j+1] = temp
9  END FUNCTION
10
11 FUNCTION timSort(arr, n, RUN = 32)
12     // Sort small runs using insertion sort
13     FOR start FROM 0 TO n-1 STEP RUN
14         end = MIN(start + RUN - 1, n - 1)
15         insertionSort(arr, start, end)
16
17     // Merge runs, doubling the merged size each pass
18     size = RUN
19     WHILE size < n
20         FOR left FROM 0 TO n-1 STEP 2 * size
21             mid = left + size - 1
22             right = MIN(left + 2*size - 1, n - 1)
23             IF mid < right
24                 merge(arr, left, mid, right) // use merge that merges two sorted halves
25             size = 2 * size
26 END FUNCTION
```

Figure 1: TimSort pseudocode

Theoretical complexity:

Time (best/avg/worst):  $O(n)$  /  $O(n \log n)$  /  $O(n \log n)$ .

Space:

$O(n)$  (uses temporary runs during merge).

Memory usage:

$O(n)$  auxiliary memory.

Stable: Yes.

## Algorithm 2: Radix Sort

Main idea:

Sort numbers digit by digit, from least significant to most significant.  
Use a stable sort (usually counting sort) for each digit.  
Handle negative and positive numbers separately, then combine.  
Does not compare elements directly.

Pseudocode:

```
1  FUNCTION radixSort(arr, n)
2      IF  $n \leq 1$  RETURN
3
4      neg  $\leftarrow$  empty list
5      pos  $\leftarrow$  empty list
6
7      // Separate negatives and positives
8      FOR each x IN arr
9          IF  $x < 0$ 
10             neg.append(-x)    // store absolute value
11          ELSE
12             pos.append(x)
13
14      // Sort both lists using LSD radix sort
15      radixLSD(neg)
16      radixLSD(pos)
17
18      // Rebuild array: negatives (reversed), then positives
19      index  $\leftarrow 0$ 
20      FOR i FROM length(neg)-1 DOWNTO 0
21         arr[index++]  $\leftarrow$  -neg[i]
22      FOR i FROM 0 TO length(pos)-1
23         arr[index++]  $\leftarrow$  pos[i]
24  END FUNCTION
25
26
27  FUNCTION radixLSD(a)
28      maxVal  $\leftarrow$  maximum element of a
29      exp  $\leftarrow 1$ 
30      WHILE maxVal / exp > 0
31         stableCountingSortByDigit(a, exp)
32         exp  $\leftarrow$  exp  $\times$  10
33  END FUNCTION
```

Figure 2: Radix Sort pseudocode

Theoretical complexity:

Time (best/avg/worst):  $O(nk)$  /  $O(nk)$  /  $O(nk)$ , where  $k$  is the number of digits.

Space:

$O(n + k)$  (counting array plus output array).

Memory usage:

$O(n + k)$  auxiliary memory.

Stable: Yes (when counting sort is stable).

### Algorithm 3: Quick Sort

Main idea:

Choose a pivot element.  
Partition the array into elements smaller than and greater than the pivot.  
Recursively apply the same process to the subarrays.  
Uses divide-and-conquer.

Pseudocode:

```
1  FUNCTION partition(arr, low, high)
2      pivot = arr[high]
3      i = low - 1
4      FOR j FROM low TO high - 1
5          IF arr[j] <= pivot
6              i = i + 1
7              SWAP arr[i] WITH arr[j]
8      SWAP arr[i+1] WITH arr[high]
9      RETURN i+1
10 END FUNCTION
11
12 FUNCTION quickSort(arr, low, high)
13     IF low < high
14         pi = partition(arr, low, high)
15         quickSort(arr, low, pi - 1)
16         quickSort(arr, pi + 1, high)
17 END FUNCTION
```

Figure 3: Quick Sort pseudocode

Theoretical complexity:

Time (best/avg/worst):  $O(n \log n)$  /  $O(n \log n)$  /  $O(n^2)$ .

Space:

$O(\log n)$  expected (recursion stack).

Memory usage:

$O(\log n)$  auxiliary memory.

Stable: No.

#### Algorithm 4: Selection Sort

Main idea:

Repeatedly find the minimum element in the unsorted part.  
Swap it with the first unsorted position.  
Grow the sorted portion one element at a time.

Pseudocode:

```
1  FUNCTION selectionSort(arr, n)
2      FOR i FROM 0 TO n-2
3          min_idx = i
4          FOR j FROM i+1 TO n-1
5              IF arr[j] < arr[min_idx]
6                  min_idx = j
7              // swap arr[i] and arr[min_idx]
8              SWAP arr[i] WITH arr[min_idx]
9  END FUNCTION
```

Figure 4: Selection Sort pseudocode

Theoretical complexity:

Time (best/avg/worst):  $O(n^2)$  /  $O(n^2)$  /  $O(n^2)$ .

Space:

$O(1)$  (in-place).

Memory usage:

$O(1)$  auxiliary memory.

Stable: No (standard implementation).



### Algorithm 5: Merge Sort

Main idea:

Divide the array into two halves.  
Recursively sort each half.  
Merge the two sorted halves into one sorted array.  
Based on divide-and-conquer.

Pseudocode:

```
1  FUNCTION merge(arr, l, m, r)
2      n1 = m - l + 1
3      n2 = r - m
4      L = copy arr[l .. m]
5      R = copy arr[m+1 .. r]
6      i = 0; j = 0; k = l
7      WHILE i < n1 AND j < n2
8          IF L[i] <= R[j]
9              arr[k] = L[i]; i = i + 1
10         ELSE
11             arr[k] = R[j]; j = j + 1
12         k = k + 1
13     WHILE i < n1
14         arr[k] = L[i]; i = i + 1; k = k + 1
15     WHILE j < n2
16         arr[k] = R[j]; j = j + 1; k = k + 1
17     END FUNCTION
18
19  FUNCTION mergeSort(arr, l, r)
20      IF l < r
21          m = l + (r - l) / 2
22          mergeSort(arr, l, m)
23          mergeSort(arr, m + 1, r)
24          merge(arr, l, m, r)
25  END FUNCTION
```

Figure 5: Merge Sort pseudocode

Theoretical complexity:

Time (best/avg/worst):  $O(n \log n)$  /  $O(n \log n)$  /  $O(n \log n)$ .

Space:

$O(n)$  (temporary arrays during merge).

Memory usage:

$O(n)$  auxiliary memory.

Stable: Yes.

## 4 Experimental Results

### 4.1 Random Input

Table 1: Execution Time (microsec) for Random Input

| Algorithm      | 100 | 1k   | 10k   | 100k    | 1M        |
|----------------|-----|------|-------|---------|-----------|
| Tim Sort       | 66  | 292  | 1218  | 17413   | 185629    |
| Radix Sort     | 64  | 858  | 4606  | 22642   | 211551    |
| Quick sort     | 16  | 171  | 967   | 13415   | 144262    |
| Selection Sort | 36  | 2074 | 81392 | 6590798 | 660000000 |
| Merge Sort     | 89  | 551  | 4271  | 24802   | 261667    |

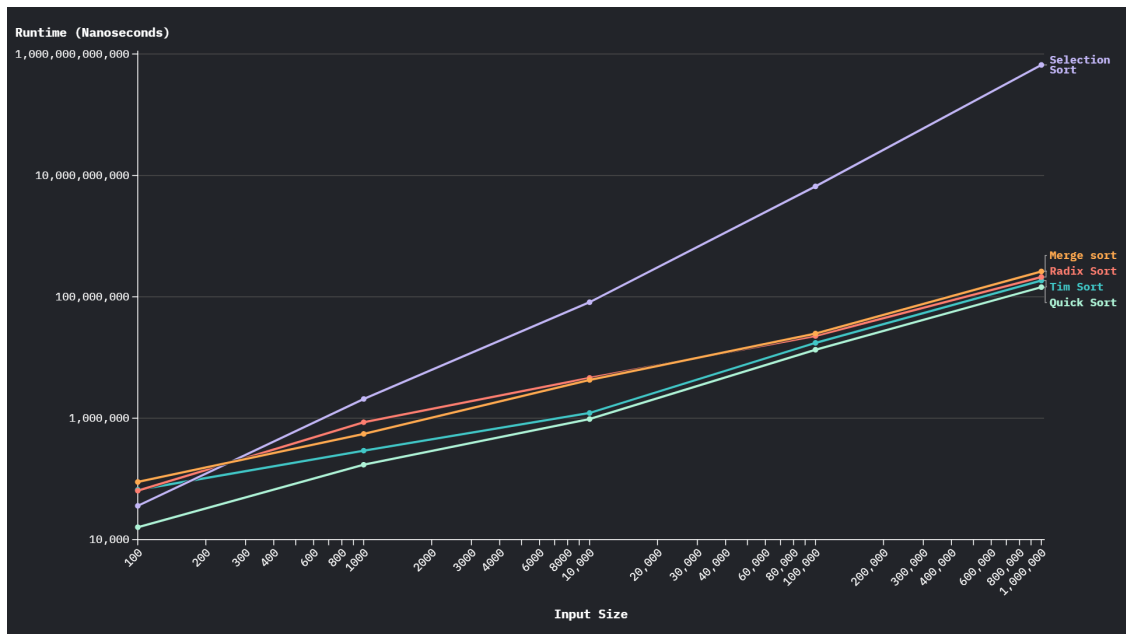


Figure 6: Execution time for Random Input

Table 2: Memory usage for Random Input (in KB)

| Algorithm      | 100 | 1k  | 10k | 100k | 1M    |
|----------------|-----|-----|-----|------|-------|
| Tim Sort       | 592 | 632 | 764 | 2088 | 13088 |
| Radix Sort     | 600 | 612 | 736 | 1784 | 12168 |
| Quick sort     | 576 | 600 | 644 | 1048 | 4956  |
| Selection Sort | 584 | 608 | 656 | 1048 | 4563  |
| Merge Sort     | 604 | 656 | 868 | 2228 | 13620 |

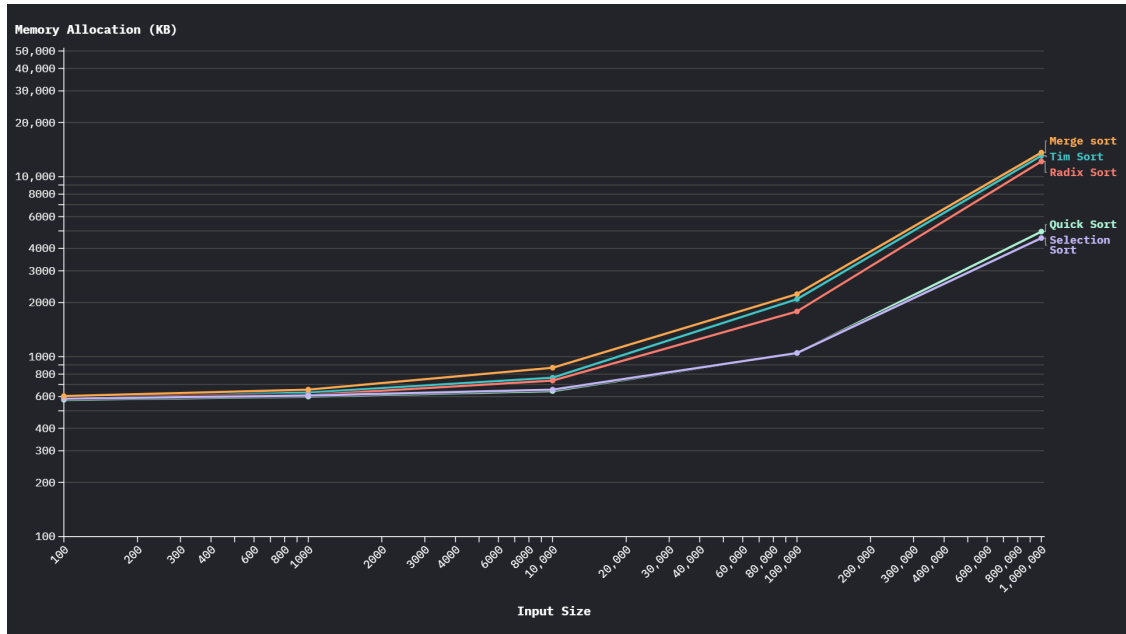


Figure 7: Memory usage for Random Input

## 4.2 Ascending Input

Table 3: Execution Time (microsec) for Ascending Input

| Algorithm      | 100 | 1k   | 10k    | 100k     | 1M         |
|----------------|-----|------|--------|----------|------------|
| Tim Sort       | 47  | 153  | 1053   | 9237     | 66517      |
| Radix Sort     | 70  | 551  | 4739   | 22942    | 211423     |
| Quick sort     | 56  | 4755 | 185616 | 16049539 | 1604953900 |
| Selection Sort | 22  | 1930 | 80709  | 7032476  | 703247600  |
| Merge Sort     | 115 | 337  | 3396   | 18096    | 155159     |

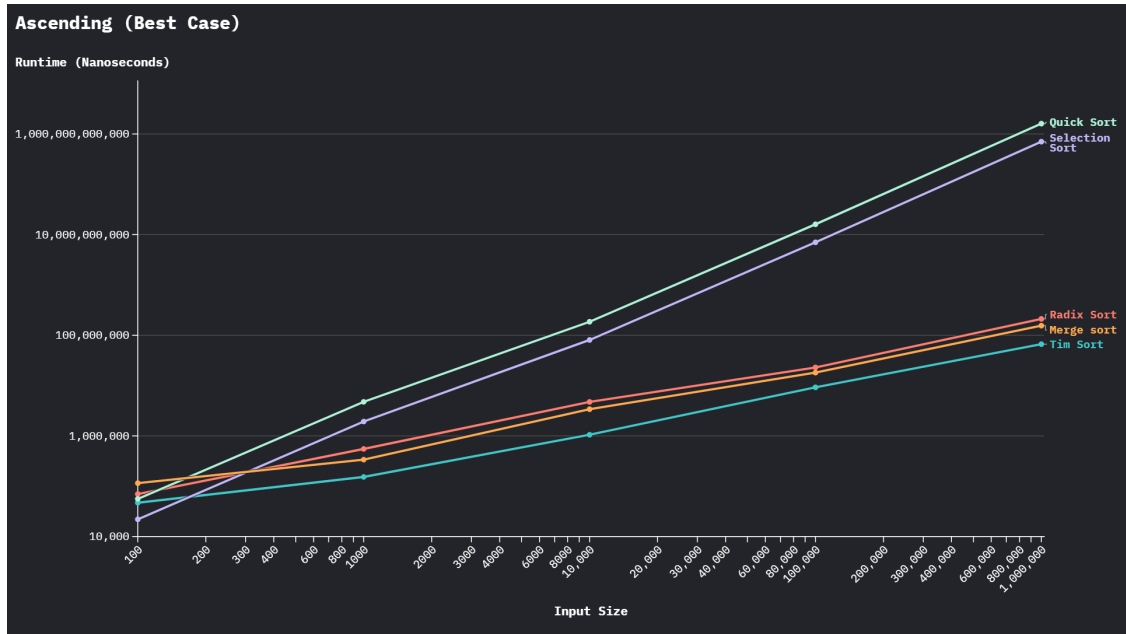


Figure 8: Execution time for Ascending Input

Table 4: Memory usage for Ascending Input (in KB)

| Algorithm      | 100 | 1k  | 10k  | 100k | 1M    |
|----------------|-----|-----|------|------|-------|
| Tim Sort       | 600 | 664 | 788  | 2136 | 13236 |
| Radix Sort     | 580 | 612 | 740  | 1800 | 10960 |
| Quick sort     | 576 | 644 | 1116 | 5628 | 28145 |
| Selection Sort | 576 | 596 | 648  | 1040 | 1440  |
| Merge Sort     | 604 | 612 | 760  | 2372 | 13584 |

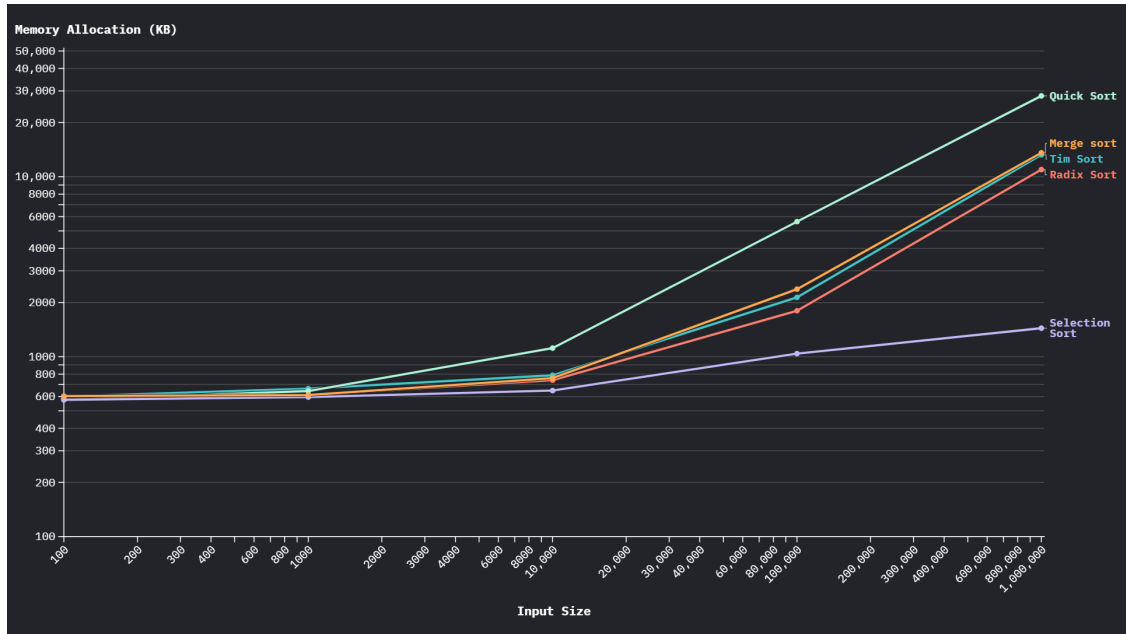


Figure 9: Memory usage for Ascending Input

### 4.3 Descending Input

Table 5: Execution Time (microsec) for Descending Input

| Algorithm      | 100 | 1k   | 10k    | 100k     | 1M        |
|----------------|-----|------|--------|----------|-----------|
| Tim Sort       | 65  | 222  | 1752   | 12197    | 103797    |
| Radix Sort     | 75  | 562  | 4490   | 22728    | 209177    |
| Quick sort     | 58  | 2884 | 129782 | 10837117 | 650227020 |
| Selection Sort | 27  | 2078 | 85280  | 7011029  | 490772030 |
| Merge Sort     | 46  | 595  | 3007   | 18015    | 157352    |

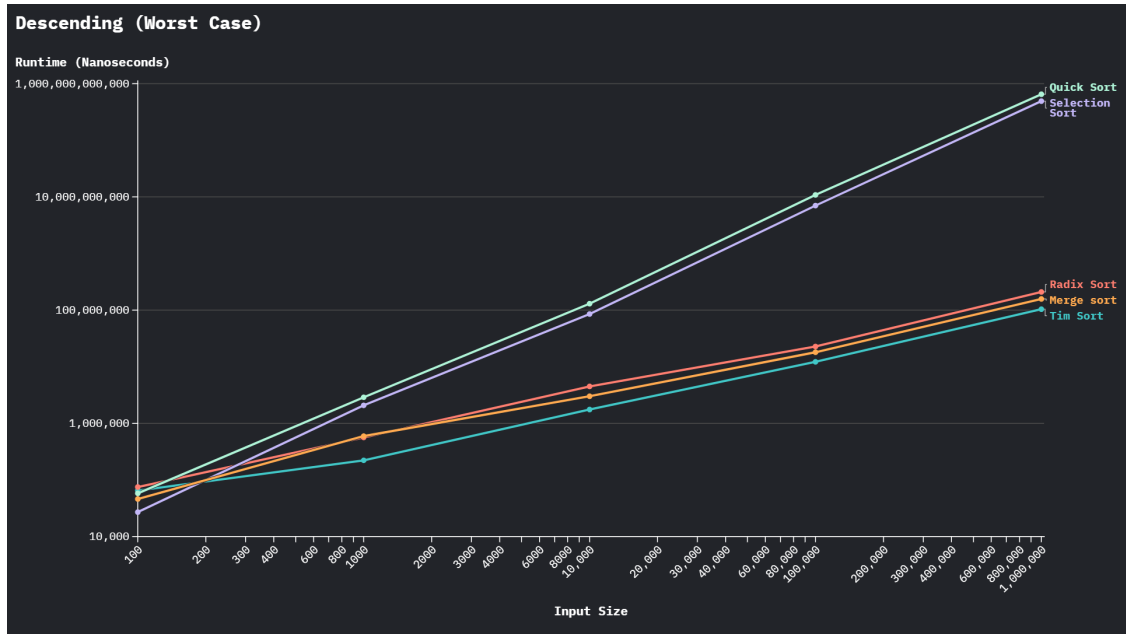


Figure 10: Execution time for Descending Input

Table 6: Memory usage for Descending Input (in KB)

| Algorithm      | 100 | 1k  | 10k  | 100k | 1M    |
|----------------|-----|-----|------|------|-------|
| Tim Sort       | 592 | 632 | 760  | 2124 | 13072 |
| Radix Sort     | 588 | 616 | 688  | 1760 | 12144 |
| Quick sort     | 576 | 628 | 1096 | 5716 | 28726 |
| Selection Sort | 564 | 580 | 624  | 1028 | 1432  |
| Merge Sort     | 572 | 648 | 800  | 2044 | 13492 |

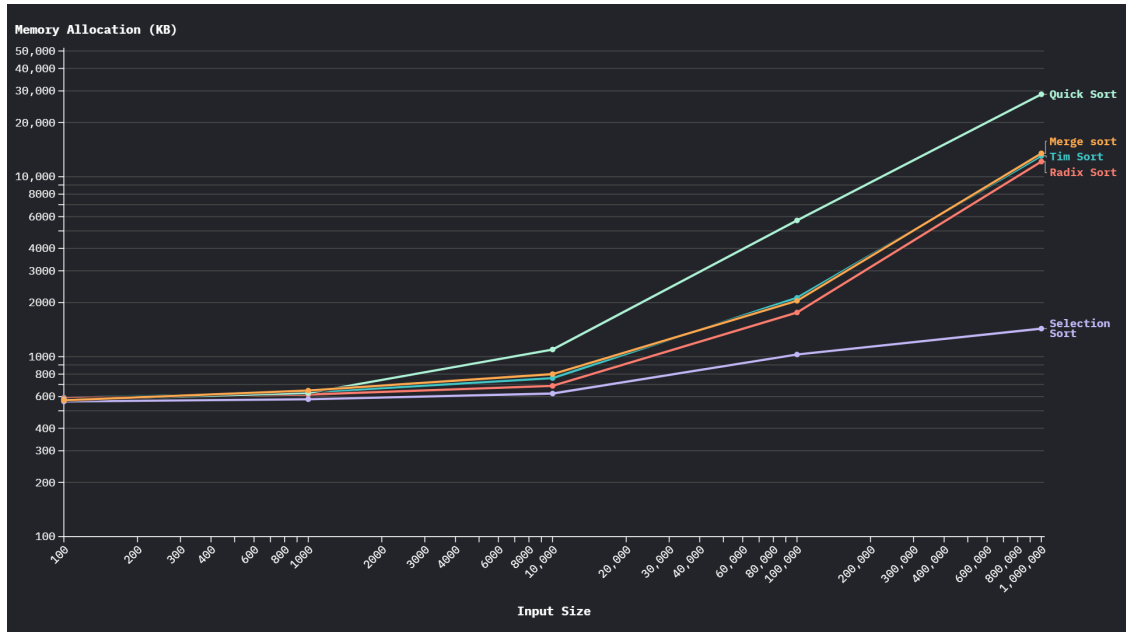


Figure 11: Memory usage for Descending Input

#### 4.4 Discussion and Observations

The experimental results confirm the theoretical complexity of the algorithms:

- **Selection Sort** shows quadratic growth in execution time, becoming impractical for large inputs, despite relatively low memory usage.
- **Quick Sort** generally performs well, but its execution time is highly sensitive to input ordering. For ascending and descending inputs, performance degrades significantly due to poor pivot choices.
- **Tim Sort** benefits from partially ordered data, showing faster execution on ascending inputs compared to random or descending ones.
- **Merge Sort** maintains stable performance across all input types, consistent with its  $O(n \log n)$  complexity, but requires more memory due to auxiliary arrays.
- **Radix Sort** demonstrates linear-time behavior relative to input size, but memory usage grows with larger datasets.

Input ordering clearly influences performance:

- Ascending inputs favor Tim Sort, which exploits runs in sorted data.
- Descending inputs penalize Quick Sort, highlighting its worst-case behavior.
- Random inputs provide a balanced view, aligning with average-case complexities.

Unexpected behaviors include:

- Quick Sort's extreme slowdown on ordered inputs, illustrating the importance of pivot selection strategies.
- Tim Sort's adaptability, which makes it superior in real-world scenarios where data is often partially sorted.

## 5 Real-World Applications

Sorting algorithms are widely used in real-world software systems, where performance, memory usage, stability, and robustness under different input distributions are critical factors. The choice of a sorting algorithm is therefore determined not only by its theoretical time complexity, but also by the practical characteristics of the application and the execution environment.

TimSort is commonly employed in applications where data is partially ordered and where stable sorting is required. Typical use cases include sorting database query results, ordering records in programming language standard libraries, and organizing data in graphical user interfaces such as file explorers or contact lists. Its ability to exploit existing order in data makes it particularly effective in real-world scenarios.

QuickSort is frequently used in performance-critical, in-memory systems due to its excellent average-case efficiency and low memory overhead. It is suitable for scientific computing, real-time analytics, game engines, and system-level utilities. However, its sensitivity to input ordering requires careful implementation to avoid worst-case performance.

Merge Sort is preferred in applications that demand predictable performance and stability regardless of input distribution. It is extensively used for external sorting of large datasets stored on disk, in data processing pipelines, and in distributed or parallel computing systems. Its main limitation is the additional memory required for merging.

Radix Sort is applied in scenarios involving integer keys or fixed-length data, where comparison-based sorting is inefficient. Practical applications include sorting identifiers, categorical data, network routing tables, and large-scale analytics with bounded key sizes. While efficient in time, its memory consumption increases with the size of the dataset.

Selection Sort has limited applicability in modern systems due to its quadratic time complexity. Nevertheless, it remains relevant in extremely memory-constrained environments, such as embedded systems, or in situations where minimizing write operations is critical. It is also commonly used for educational purposes and for sorting very small datasets.

### 5.1 Comparative Characteristics

Table 7: Theoretical Characteristics of the Analyzed Algorithms

| Algorithm      | Avg. Time     | Worst Time    | Extra Space | Stable |
|----------------|---------------|---------------|-------------|--------|
| Tim Sort       | $O(n \log n)$ | $O(n \log n)$ | $O(n)$      | Yes    |
| Quick Sort     | $O(n \log n)$ | $O(n^2)$      | $O(\log n)$ | No     |
| Merge Sort     | $O(n \log n)$ | $O(n \log n)$ | $O(n)$      | Yes    |
| Radix Sort     | $O(nk)$       | $O(nk)$       | $O(n + k)$  | Yes    |
| Selection Sort | $O(n^2)$      | $O(n^2)$      | $O(1)$      | No     |

### 5.2 Application-Oriented Analysis

#### TimSort

TimSort is particularly appropriate in environments where data is partially ordered or where stability is mandatory. Many real-world datasets, such as database query results, user interface lists, and log files, exhibit natural runs (pre-existing sorted segments). Because TimSort detects and exploits these runs, it performs exceptionally well in such cases.

Its stability makes it ideal for multi-key sorting tasks, for example sorting records first by one attribute and then by another while preserving relative order. However, its  $O(n)$  auxiliary memory requirement may be restrictive in memory-constrained systems. Experimental results confirm that TimSort performs best on ascending inputs, exploiting natural order.

#### QuickSort

QuickSort is often preferred in performance-critical in-memory systems where average-case speed is the dominant criterion and stability is not required. Due to good cache locality and low constant factors, it frequently outperforms other comparison-based algorithms in practice.

However, in scenarios where worst-case guarantees are required (e.g., real-time systems), its potential  $O(n^2)$  degradation must be considered. Modern implementations mitigate this risk through improved pivot selection strategies. In experiments, QuickSort showed excellent performance on random inputs but degraded significantly on ascending and descending inputs due to poor pivot choices.



## Merge Sort

Merge Sort is suitable for applications requiring guaranteed  $O(n \log n)$  performance independent of input ordering. It is particularly effective in external sorting scenarios where data resides on disk and must be processed in blocks.

Because it is stable and performs well on linked lists, it is often used in large-scale data processing systems. The primary limitation is its linear auxiliary memory consumption. Experimental data shows consistent performance across all input types, confirming its theoretical predictability.

## Radix Sort

Radix Sort is appropriate when sorting integer or fixed-length string data, especially when the range of values ( $k$ ) is not excessively large. It achieves near-linear performance  $O(nk)$  and is stable, making it suitable for applications like sorting IDs, categorical data, or large datasets with bounded keys.

Its memory usage grows with the number of buckets required, which can be restrictive for very large ranges. In practice, Radix Sort demonstrated competitive execution times but required significant auxiliary memory, especially for large inputs.

## Selection Sort

Selection Sort is primarily of educational and theoretical interest due to its simplicity. It has quadratic time complexity  $O(n^2)$ , making it impractical for large datasets. It is not stable, but requires only constant extra space  $O(1)$ .

### Best use cases:

- **Extremely memory-constrained systems** where auxiliary memory is nearly non-existent.
- **Minimizing write operations** when the cost of swapping/writing to memory is very high (e.g., EEPROM or Flash memory).
- **Practical examples:** simple embedded sensors that only need to sort a few dozen data points; educational demonstrations of nested loops and algorithmic logic.

**Why suitable:** our report shows Selection Sort has constant  $O(1)$  space complexity. While it was the slowest in our tests (taking 11.23 seconds for 100,000 elements), it performs the minimum possible number of swaps ( $n$ ), making it gentle on hardware with limited write cycles.

## 5.3 Context-Based Recommendation Table

Table 8: Recommended Usage Contexts

| Scenario                                       | Recommended Algorithm                |
|------------------------------------------------|--------------------------------------|
| Partially sorted or nearly ordered data        | TimSort                              |
| Multi-key stable sorting                       | TimSort or Merge Sort                |
| Maximum average speed (in-memory, random data) | QuickSort                            |
| Guaranteed worst-case performance              | Merge Sort                           |
| Minimal extra memory required                  | Selection Sort (small datasets only) |
| Large integer or categorical datasets          | Radix Sort                           |
| Educational or very small datasets             | Selection Sort                       |

## 5.4 General Insight

There is no universally optimal sorting algorithm. The correct choice depends on data size, distribution, stability requirements, and memory constraints. Experimental results from this study confirm that theoretical complexity

alone is insufficient to determine practical performance; input characteristics and implementation details significantly influence outcomes.

TimSort excels when data is partially ordered, QuickSort dominates on random inputs but suffers on ordered ones, Merge Sort provides predictable performance with stability guarantees, Radix Sort is efficient for integer data with bounded ranges, and Selection Sort remains useful only for very small datasets or educational purposes.

## 6 Competition: The Fastest Sorting Algorithm

### 6.1 Scenario

The competition experiment evaluates the performance of five sorting algorithms implemented in C:

- Tim Sort
- Radix Sort
- Quick Sort
- Selection Sort
- Merge Sort

Each algorithm is tested using predefined input datasets stored in external files.

**Organization of Input Data.** In contradistinction to dynamic generation during runtime, the input data is pre-generated and stored in text files located in the `inputs/` directory. Each file name follows the structure:

`inputs/input_<N>_<order>.txt`

where:

- `<N>` represents the number of elements (100, 1000, 10000, 100000, 1000000)
- `<order>` represents the initial arrangement: `ascendent`, `descendent`, `random`

Example:

`inputs/input_1000000_random.txt`

This approach guarantees identical input across all algorithms, reproducibility of results, and controlled testing conditions.

**File Format.** Each input file contains exactly  $N$  integers, separated by whitespace (spaces or line breaks).

Example:

`45 12 78 3 90 6 34`

The file does not include the value of  $N$  inside it. The number of elements is specified manually by the user before execution.

**Data Loading Procedure.** The program performs the following steps:

- The user selects the sorting algorithm, number of elements, and initial order type.
- The corresponding file name is constructed using `sprintf()`.
- The file is opened in read mode.
- Memory is dynamically allocated using `malloc`.
- The integers are loaded sequentially into the allocated array using `fscanf`.
- The file is closed before sorting begins.

Dynamic memory allocation ensures that large datasets (up to 1,000,000 elements) are handled efficiently.

In the context of the final competitive scenario, the primary benchmark is the process of sorting an array of 1,000,000 randomly ordered elements. It is imperative to note that all algorithms are subjected to rigorous testing in controlled conditions: same machine, same compiler, unchanged dataset, and the same memory allocation strategy. The objective of this scenario is to ascertain which algorithm exhibits the optimal practical performance on large-scale random input.

## 6.2 Performance Metrics

In this competition, two distinct performance measurements were defined in order to evaluate the efficiency of the sorting algorithms:

1. **Sorting time only** — This metric isolates the computation required to sort the elements in memory. In the implementation, the measurement begins immediately before the sorting function is invoked and stops once the algorithm has completed. The time is obtained using the Windows high-resolution performance counters (`QueryPerformanceCounter`) and reflects the pure algorithmic efficiency without any influence from input/output operations.
2. **Sorting time plus file output time** — This metric includes both the sorting computation and the time required to write the sorted elements to the specified output destination. In the program, after the sorting step is completed, the results are either printed to the console or written to a file. The measurement continues until all output operations are finished. The total time is again measured using performance counters, combining the sorting duration with the additional overhead introduced by I/O.

By distinguishing between these two measurements, the evaluation separates the intrinsic computational complexity of the algorithms from the practical overhead of storing or displaying results. This approach provides both a theoretical benchmark (sorting only) and a realistic assessment of performance in real-world scenarios (sorting plus output).

## 6.3 Command-Line Arguments

Command-line arguments allow the program to receive execution parameters directly from the terminal, without modifying the source code. In C, these parameters are accessed through the function signature `int main(int argc, char *argv[])`, where `argc` represents the number of arguments and `argv` contains their values.

In this sorting benchmark program, command-line arguments are used to automate execution. If no arguments are provided (`argc == 1`), the program runs in interactive mode using the `standart_case()` function. Otherwise, the program executes `argument_case(argv)`, which reads the provided parameters and configures the execution automatically.

The arguments specify the sorting algorithm, number of elements, initial array order, and output destination. Based on these values, the program loads the corresponding input file, allocates memory dynamically, executes the selected sorting algorithm, and outputs the results.

Example executions:

```
./sort 3 1000000 3 1 # QuickSort, 1,000,000 random elements, output to console
./sort 1 1000000 2 2 # TimSort, 1,000,000 descending elements, output to file
./sort 5 10000 1 1 # Merge Sort, 10,000 ascending elements, output to console
```

This approach improves flexibility and allows efficient automated benchmarking of sorting algorithms under controlled experimental conditions.

## 6.4 Competition Results

Table 9: Competition Results for 1,000,000 Random Elements

| Algorithm      | Sort Time (s) | Sort + I/O (s) |
|----------------|---------------|----------------|
| TimSort        | 0.233314      | 0.757410       |
| Radix Sort     | 0.253963      | 0.782472       |
| QuickSort      | 0.188493      | 0.666018       |
| Selection Sort | < 5000        | ~ 5000         |
| Merge Sort     | 0.404814      | 1.090908       |

QuickSort achieves the best performance in both metrics, with the lowest execution time and the lowest sort+I/O time, likely due to its efficient average-case behavior and low overhead on random input. TimSort and Radix Sort follow closely, while Merge Sort is slower because of its additional merging overhead and auxiliary memory usage. Selection Sort is orders of magnitude slower, confirming its quadratic complexity and poor scalability for large inputs.

## 7 Conclusion

This research presents a systematic empirical analysis of five well-known sorting algorithms (Tim Sort, Radix Sort, Quick Sort, Selection Sort, and Merge Sort), examining their performance across different sizes and distributions of input data. By narrowing the gap between theoretical  $O$ -notation and practical implementation, we have identified key compromises that determine algorithm selection in real-world software engineering.

Our experimental data confirms that there is no universal solution for sorting. The performance of each algorithm depends heavily on the nature of the input data:

- **Best suited for general/random data:** Quick Sort remains the performance leader for in-memory sorting due to low constant factors and excellent cache locality, although Radix Sort offers a competitive alternative without comparison for integer datasets.
- **Best suited for partially sorted data:** Tim Sort greatly outperforms other algorithms when the input data is already sorted in ascending order or contains runs, efficiently using its hybrid insert-merge logic to achieve near-linear performance.
- **Best option for stability:** When preserving the relative order of equal elements (multi-key sorting), the most reliable options are Tim Sort and Merge Sort.
- **Worst scalability:** Selection Sort demonstrated the expected quadratic growth ( $O(n^2)$ ), making it impractical for datasets longer than 10,000 elements, regardless of input distribution.

**The impact of input order and memory.** Input distribution can significantly affect execution time. While Merge Sort remains stable for all distributions, Quick Sort is vulnerable to degraded performance in worst-case scenarios, and Tim Sort performs best when a pre-order is available. In terms of memory, there is a clear difference between algorithms that operate in place and algorithms that require additional space. Selection Sort and Quick Sort are memory-efficient ( $O(1)$  and  $O(\log n)$ , respectively), whereas Merge Sort and Radix Sort require significant additional memory ( $O(n)$ ), which can create a bottleneck in limited-memory environments such as embedded systems.

**Limitations.** Although the methodology provided a structured comparison, some limitations were noted: each configuration was run once per run, which led to potential noise from OS scheduling and background processes; results may vary depending on architecture, as seen in the transition from the Intel-based MacBook Pro to the AMD-based ASUS laptop; and the study focused specifically on integer arrays.

Finally, this analysis provides a practical guide for choosing sorting strategies based on the specific requirements of the application, depending on whether speed, memory efficiency, or data stability is most important in the given case.

## References

## References

- [1] Codefinity. The importance and evolution of sorting algorithms, 2024. URL <https://codefinity.com/blog/The-Importance-and-Evolution-of-Sorting-Algorithms>. Accessed 2026-02-26.
- [2] Jane Doe. *Placeholder Reference for Draft Bibliography*. Example Press, 2024.
- [3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Introduction to algorithms, 2009. URL <https://mitpress.mit.edu/9780262046305/introduction-to-algorithms>. Accessed 2026-02-26.
- [4] Society for Industrial and Applied Mathematics. Siam journal on computing (article page), 2010. URL <http://epubs.siam.org/doi/abs/10.1137/100785351?journalCode=smjcat>. Accessed 2026-02-26.
- [5] Robert Sedgwick and Kevin Wayne. Algorithms, 4th edition, 2011. URL <https://algs4.cs.princeton.edu/home/>. Accessed 2026-02-26.
- [6] Schloss Dagstuhl. Lipics.esa.2018.4 (dagstuhl), 2018. URL <https://drops.dagstuhl.de/entities/document/10.4230/LIPICs.ESA.2018.4>. Accessed 2026-02-26.
- [7] Semantic Scholar. Semantic scholar pdf (bf99/722d1c...), 2010. URL <https://pdfs.semanticscholar.org/bf99/722d1cad87c231667fe21c69ab63aa3df4e3.pdf>. Accessed 2026-02-26.
- [8] Informatica Journal. Informatica journal article (id 4752), 2020. URL <https://www.informatica.si/index.php/informatica/article/view/4752/3673>. Accessed 2026-02-26.

- [9] National Center for Biotechnology Information. Pubmed central article (pmc12867261), 2024. URL <https://pmc.ncbi.nlm.nih.gov/articles/PMC12867261/>. Accessed 2026-02-26.
- [10] National Center for Biotechnology Information. Pubmed central article (pmc12863692), 2024. URL <https://pmc.ncbi.nlm.nih.gov/articles/PMC12863692/>. Accessed 2026-02-26.
- [11] Linux man-pages project. `getrusage(2)` — linux manual page, 2024. URL <https://man7.org/linux/man-pages/man2/getrusage.2.html>. Accessed 2026-02-26.